数学与 CS 问答笔记

math-answer-note

按日归档的问答笔记：三问反思法（关键点/变式/同类归类）+ 讲课叙事流 + CeTZ 图示，结构与课程笔记同构。

October 07, 2026

Contents

1. 当日起点：项目状态与一处待修的语义 3
   1.1. 进度概览 3
   1.2. 一处需要修的问题：Get 的 ok 永远是 false 3
   1.3. 索引与速查（本次新增） 4
   1.4. 待办 4
2. `m[k]` 在运行时到底做了什么 5
   2.1. 费曼直觉 5
   2.2. 硬核机制 5
      2.2.1. 第一步：编译后调用了谁 5
      2.2.2. 第二步：单值只是双值的包装 5
      2.2.3. 第三步：主流程的六步 5
      2.2.4. 四个值得单独记住的设计 6
      2.2.5. 顺带撞出的一个反例：`return m[key]` 编译不过 7
   2.3. 陷阱与易错 7
   2.4. 关键点 7
   2.5. 变式检验 8
   2.6. 同类归类 8
   2.7. Go 语言机制补充 8
      2.7.1. 为什么叫 “Swiss table” 8
      2.7.2. 长字符串的额外优化 8
3. `defer kv.Close()` 是什么意思 10
   3.1. 费曼直觉 10
   3.2. 硬核机制 10
      3.2.1. 规范怎么说（Defer statements 一节） 10
      3.2.2. 实测：五种行为一次看清 11
      3.2.3. 回到你的这行代码 12
      3.2.4. `defer` 的三个常见坑 12
   3.3. 陷阱与易错 12
   3.4. 关键点 13
   3.5. 变式检验 13
   3.6. 同类归类 13
   3.7. Go 语言机制补充 13
      3.7.1. defer 执行时机的精确定义 13
      3.7.2. defer、panic、recover 三者的关系 14

Index of Listings 15

1. 当日起点：项目状态与一处待修的语义

这一节记录 10-07 开工时的状态（距上次 10-04 已过 3 天），作为今天的基线。

1.1. 进度概览

项	状态
`internal/db/kv.go`	`Get` 与 `Set` 都有函数体了；`Del` 还只有签名 → 编译停在 `missing function body`
`Get` 的语义	**有一处需要修**：`ok` 恒为 `false`（见下节）
`Set` 的语义	暂时 `return true, nil`（写死了，与课程的 `updated` 语义不同，但当前测试恰好能过）
测试	补上 `Del` 后可编译，但 `TestKVBasic` 会在第 19 行失败——原因见下节

1.2. 一处需要修的问题：Get 的 ok 永远是 false

你现在的写法用了 **map 索引的单值形式**：

```
func (kv *KV) Get(key []byte) (val []byte, ok bool, err error) {
  val = kv.mem[string(key)]   // 单值形式：只取到「值」，拿不到「是否存在」
  return val, ok, err
}
```

Listing 1: 能编译，但 ok 永远是零值（false）

单值形式只返回“值”（键不存在时给零值），**不会**顺带告诉你键在不在；要拿到那个布尔，必须用 comma-ok 双值形式：

```
func (kv *KV) Get(key []byte) (val []byte, ok bool, err error) {
  val, ok = kv.mem[string(key)]   // 双值形式：同时拿到「值」和「是否存在」
  return
}
```

Listing 2: 正确写法：双值接双值；err 保持零值 nil

实测后果——我在副本里补上 `Del` 让包能编译，然后跑测试：

```
$ go test ./internal/db/
--- FAIL: TestKVBasic (0.00s)
    kv_test.go:19:
          Error:        Should be true
          Test:       TestKVBasic
FAIL
```

Listing 3: 第 19 行正是 assert.True(t, string(val) == “v1” && ok && err == nil)——ok 为 false，断言失败

📌 **核心要点** `val = kv.mem[k]` 与 `val, ok = kv.mem[k]` 只差一个变量，语义差一个“存在性”。前者取不到键时安静地给零值（`nil`），后者额外告诉你“这个键到底在不在”。凡是需要区分“键不存在”与“存了一个空值”的场景，都必须用双值形式。

1.3. 索引与速查（本次新增）

本目录已累积 5 份文件、23 个主题，“我记得写过但找不到”开始成为问题，所以生成了一份索引：

```
$ ls note/
INDEX.md          ← 新增：高频主题速查表 + 按日期分节的完整清单
2026-09-30.typ   2026-10-01.typ   2026-10-03.typ   2026-10-04.typ   2026-10-07.typ
```

Listing 4: INDEX.md 由脚本扫描各 .typ 的一级标题自动生成，追加新问答后重新跑一次即可刷新

📌 **核心要点** 查**内置类型**（map / slice / chan）文档的三条路径，最容易忘的是第一条：**`go doc builtin`**——预声明标识符的总入口，`make`、`delete`、`clear`、`len`、`cap` 都在这里。注意两个反直觉的失败：`go doc map` 会被当成包路径（报 `no Go files`），`go doc builtin.map` 也查不到（`map` 是**类型构造器**，`builtin` 里没有它的条目）。完整版见 10-04 的《怎么用 go doc 查 map 相关的内容》。

1.4. 待办

1. **修 `Get`**：把单值形式改成 `val, ok = kv.mem[string(key)]`（这是当前测试失败的根源）；
2. **补 `Del`**：目前只有签名，编译报 `missing function body`（提示：`delete` 对不存在的键是安全空操作，所以要先查一次拿到 `deleted`）；
3. **可选**：把 `Set` 的 `return true, nil` 改成课程语义的 `updated = !exist || !bytes.Equal(prev, val)`（10-01 那节有完整依据）；
4. 全部补完后按四步验证：`gofmt -l .` → `go build ./...` → `go vet ./...` → `go test ./...`。

2. `m[k]` 在运行时到底做了什么

2.1. 费曼直觉

一句话概括：**算一次哈希 → 用它定位到一个“组” → 组内先用 1 字节“指纹”快速筛，筛中了才比完整的键。**

这是 Swiss table 的核心思路：**把“比较键”这种昂贵操作，尽量用“比较 1 个字节”替代掉。**下面按“编译后调谁 → 运行时怎么走”的顺序拆。

2.2. 硬核机制

2.2.1. 第一步：编译后调用了谁

```
func single(key string) []byte { return m[key] }          // 单值形式
func commaOk(key string) ([]byte, bool) { v, ok := m[key]; return v, ok }  // 双值形式
```

Listing 5: 同一个索引表达式的两种写法

```
$ go build -gcflags=-S ./... | grep -E 'CALL runtime\.(mapaccess|mapassign)'
rel 43+4 t=R_CALL runtime.mapaccess1_faststr+0    // 单值
rel 43+4 t=R_CALL runtime.mapaccess2_faststr+0    // 双值
rel 83+4 t=R_CALL runtime.mapassign_faststr+0     // map 字面量的初始化
```

Listing 6: 后缀 `_faststr` 是编译器为 map[string]X 生成的特化版本

2.2.2. 第二步：单值只是双值的包装

实现位于 `/usr/lib/go/src/internal/runtime/maps/runtime_faststr.go`。第一件事就很有意思：

```
//go:linkname runtime_mapaccess1_faststr runtime.mapaccess1_faststr
func runtime_mapaccess1_faststr(typ *abi.MapType, m *Map, key string) unsafe.Pointer {
  p, _ := runtime_mapaccess2_faststr(typ, m, key)   // 单值 = 双值 + 丢掉 bool
  return p
}
```

Listing 7: 单值形式没有独立实现——查找逻辑完全相同，差别只在要不要那个 bool

> 📌 **核心要点** 这从**实现层**解释了前面那个结论：`v := m[k]` 拿不到“存在性”，不是因为它算了别的东西，而是因为**布尔值被丢掉了**。两种形式做的工作一模一样。

2.2.3. 第三步：主流程的六步

`runtime_mapaccess2_faststr` 的主干（去掉并发检测等旁支）：

```
    if m == nil || m.Used() == 0 {
      return unsafe.Pointer(&zeroVal[0]), false      // ① 空 map / nil map: 直接给零值
+ false
    }
    if m.writing ^= 0 {
      fatal("concurrent map read and map write")   // ② 并发写检测
    }
    if m.dirLen <= 0 {
      elem := m.getWithoutKeySmallFastStr(typ, key) // ③ 小 map（单组）走快路径
      ...
    }

    dohash:
      var hash uintptr
      if memHashAESImplemented && UseAeshash {
        hash = memHashAES(unsafe.Pointer(unsafe.StringData(key)), m.seed, uintptr(len(key)))
      } else {
        hash = memHashFallback(...)               // ④ 算哈希（AES 硬件加速，带每-map 随机
种子）
      }
      h2 := uint8(h2(hash))                        // ⑤ 取低 7 位当「指纹」
      ctrls = *g.ctrls()                           //    读 8 字节控制字
      slotKey = g.key(typ, 0)

      for i := range uintptr(abi.MapGroupSlots) { // ⑥ 遍历组内 8 个槽
        if uint8(ctrls) == h2 && key == *(*string)(slotKey) {
          ... // 先比 1 字节指纹，命中才比完整 key
        }
        ctrls >>= 8
        slotKey = unsafe.Pointer(uintptr(slotKey) + keyStride)
      }
```

Listing 8: 六步主干：空检查 → 并发检查 → 小 map 快路径 → 算哈希 → 取指纹 → 组内比对

2.2.4. 四个值得单独记住的设计

设计	做法	解决的问题
H2 指纹	哈希的低 7 位存在控制字的 1 个字节里；先比这 1 字节，命中才比完整键	组内 8 个槽里通常有 7 个只需 1 次字节比较就被排除
每 map 一个 seed	`m.seed` 参与哈希计算，每个 map 不同	防止攻击者构造“全员碰撞”的键（哈希洪泛攻击）
AES 哈希	CPU 支持时用 `memHashAES`（AES-NI 指令），否则回退到纯 Go 实现	哈希既快又分布均匀
零值指针	找不到时返回 `unsafe.Pointer(&zeroVal[0])`——一个全局零值变量的地址	让上层统一解引用，不必到处判空；这也是“读不存在的键得到零值”的落地方式

📌 **核心要点** 返回的**不是 nil，而是指向全局零值的指针**。所以 `v := m["不存在的键"]` 得到的 `v` 是那个元素类型的零值（`[]byte` 就是 `nil`），而不是某种错误状态——这正是前面讲的“map 索引不产生 error”在实现层的对应。

2.2.5. 顺带撞出的一个反例：`return m[key]` 编译不过

写探针程序时我顺手试了一下：

```
func commaOk(key string) ([]byte, bool) {
  return m[key]          // 想让 return 完成 comma-ok
}
```

Listing 9: 看起来合理，但编译失败

```
$ go build ./...
./main.go:10:9: not enough return values
  have ([]byte)
  want ([]byte, bool)
```

Listing 10: return 只拿到 1 个值——comma-ok 的“特例形式”不包括 return 语句

规范里那句 **used in an assignment statement or initialization of the special form** 是有边界的：comma-ok **只在赋值/初始化位置**生效，`return` 不行。想要双值，就得先赋值给变量：

```
func commaOk(key string) ([]byte, bool) {
  v, ok := m[key]        // 先在赋值位置取到两个值
  return v, ok
}
```

Listing 11: 绕一步即可——comma-ok 在 := 里合法

2.3. 陷阱与易错

1. **以为单值形式“更省”**。它和双值形式做的工作完全相同（源码里就是包装），只是丢掉了布尔——省不下任何开销；
2. **在 `return` 里指望 comma-ok**。见上面的实测；
3. **以为读不存在的键返回 nil 指针**。返回的是指向全局零值的指针，由编译器解引用成对应类型的零值；
4. **忽略并发检测**。同一个 map 上“读的同时有写”会触发 `fatal("concurrent map read and map write")`——这不是 panic 可以 recover 的，是直接中止程序。

2.4. 关键点

📌 **核心要点** `m[k]` 的运行时流程：**空检查 → 并发检查 → 算哈希（带 seed，AES 加速）→ 取低 7 位指纹 → 在组内 8 个槽里先比指纹、再比键**。找到就返回元素地址，找不到就返回全局零值的地址——这就是“取不到时给零值”的由来。

2.5. 变式检验

1. **换成写入** `m[k] = v`：走 `mapassign_faststr`，流程类似但末尾要写入并可能触发**扩容**（grow）；另外往 nil map 写入会 panic（规范原文：`Assigning to an element of a nil map causes a run-time panic`）；
2. **换成整数键**：调用变为 `mapaccess1_fast64` / `fast32`，实现文件是 `runtime_fast32.go` / `runtime_fast64.go`——查找骨架完全相同，只是键的比较与哈希函数不同；
3. **换成自定义结构体键**：会走通用版 `runtime_mapaccess1`，键通过 `unsafe.Pointer` 传递并按类型大小比较，没有 `_faststr` 那类特化优化。

2.6. 同类归类

1. 与散列表教科书的对照：经典链地址法把哈希值映射到“桶链表”，最坏情况退化成线性查找；Swiss table 用**开放寻址 + 控制字**，把“组内 8 个槽”的比较做成可按字节并行筛选的形式，缓存局部性也更好；
2. 与我们本项目的关系：这正是课程后面要用到的那类设计——`sorted_file.go` 的 SSTable 会用**有序数组 + 二分**解决“查找”，而内存 KV 用哈希表解决“查找”，两者都是在权衡“查找速度 / 有序性 / 插入成本”；
3. 往下延伸：`maps` 包里的 `Keys` / `Values` 返回迭代器而不是切片，也是因为**哈希表的顺序本身没有意义**——文档里那句 `The iteration order is not specified` 就是实现决定的。

2.7. Go 语言机制补充

2.7.1. 为什么叫 “Swiss table”

`internal/runtime/maps/map.go` 的包注释：

```
The map design is based on Abseil's "Swiss Table" map design
(https://abseil.io/about/design/swisstables), with additional modifications
to cover Go's additional requirements, discussed below.

Terminology:
- Slot: A storage location of a single key/element pair.
- Group: A group of abi.MapGroupSlots (8) slots, plus a control word.
- Control word: An 8-byte word which denotes whether each slot is empty,
  deleted, or used. If a slot is used, its control byte also contains the
  lower 7 bits of the hash (H2).
- H1: Upper 57 bits of a hash.
- H2: Lower 7 bits of a hash.
```

（本机 go1.27.1 用的就是这套实现，Go 1.24 起替换了旧的 `hmap` + 桶链设计。）

2.7.2. 长字符串的额外优化

小 map 路径里还有一段针对**长键**的优化，注释写得很直白：

```
// The 64 threshold was chosen based on performance of BenchmarkMapStringKeysEight,
// where there are 8 keys to check, all of which don't quick-match the lookup key.
// In that case, we can save hashing the lookup key. That savings is worth this extra code
// for strings that are long enough that hashing is expensive.
if len(key) > 64 {
  // String hashing and equality might be expensive. Do a quick check first.
```

也就是说：键超过 64 字节时，**先做一次廉价比较**（长度/指针层面的快速判断），如果 8 个槽都过不了这一关，就**连哈希都不用算**。这是“把最贵的操作放到最后”的典型工程取舍。

> 📌 **核心要点** 一句话收束：`m[k]` **的本质是“用哈希定位、用指纹筛选、用比较确认”的三段式。** 单值形式与双值形式在实现上是同一个函数，差别只是那个布尔有没有被丢掉；找不到键时返回的是全局零值的地址，而不是错误。

3. `defer kv.Close()` 是什么意思

3.1. 费曼直觉

`defer` 解决的问题很具体：**“这行清理代码，不管函数怎么结束，都必须执行”**。

没有它的时候你得在每个 `return` 前面手写一遍 `kv.Close()`——漏一个就泄漏一个资源；有了它，只需在**拿到资源之后**写一次：

```
拿到资源 → defer 注册释放 → 中间随便怎么写（return / panic 都不怕）→ 函数返回前自动释放
```

> 📌 **核心要点** `defer kv.Close()` 的含义是 **:把 `kv.Close()` 这次调用推迟到 `TestKVBasic` 返回时才执行**——不管是正常走完、中途 `return`、还是 panic。

3.2. 硬核机制

3.2.1. 规范怎么说（Defer statements 一节）

`defer` 是**关键字**，不是标准库 API，所以 `go doc` 查不到（和 `:=`、`=` 一样）。规范的原文是这样的：

```
A "defer" statement invokes a function whose execution is deferred
to the moment the surrounding function returns, either because the
surrounding function executed a return statement,
reached the end of its function body,
or because the corresponding goroutine is panicking.

DeferStmt = "defer" Expression .

The expression must be a function or method call; it cannot be parenthesized.

Each time a "defer" statement executes, the function value and parameters to the call are
evaluated as usual and saved anew but the actual function is not invoked.
Instead, deferred functions are invoked immediately before
the surrounding function returns, in the reverse order they were deferred.
```

四个要点，逐条对上：

规范说的	含义
`deferred to the moment the surrounding function returns`	推迟到**外层函数**返回时（不是语句块结束时）
三种触发时机	`return` 语句 / 走到函数体末尾 / **goroutine panic**
`must be a function or method call`	必须写成调用形式 `kv.Close()`，不能只写 `defer kv.Close`
`parameters are evaluated as usual and saved anew`	**参数在 defer 那一刻就求值并存下来了**——这是最大的坑

3.2.2. 实测：五种行为一次看清

```go
// ① 执行顺序：后进先出（LIFO）
func order() {
  defer fmt.Println("  defer 1")
  defer fmt.Println("  defer 2")
  defer fmt.Println("  defer 3")
  fmt.Println("  函数体结束")
}

// ② 参数在 defer 语句执行时就求值了
func argTiming() {
  i := 0
  defer fmt.Println("  defer 捕获 i =", i)
  i = 100
  fmt.Println("  函数体里 i =", i)
}

// ③ 闭包形式：真正执行时才读变量
func closureTiming() {
  i := 0
  defer func() { fmt.Println("  闭包读 i =", i) }()
  i = 100
}

// ④ 命名返回值可以被 defer 改写
func namedReturn() (n int) {
  defer func() { n++ }()
  return 10
}
```

Listing 12: 四段探针：顺序、参数求值、闭包、命名返回值

```
$ go run .
① LIFO 顺序:
  函数体结束
  defer 3
  defer 2
  defer 1
② 参数求值时机:
  函数体里 i = 100
  defer 捕获 i = 0        ← 参数在 defer 时就定格了
③ 闭包 vs 参数:
  闭包读 i = 100          ← 闭包是执行时才读
④ 命名返回值: 11          ← defer 改掉了返回值
⑤ panic 情形:
  panic 前注册的 defer 仍然执行
  recover 到: 出事了
```

Listing 13: 五条结论一次跑齐；②与③的差别是 defer 最经典的坑

⚠ **陷阱预警** 参数在 `defer` 语句执行的那一刻就被求值并保存了（测② 输出 `0` 而不是 `100`）。想让读取发生在**真正执行时**，得包成闭包：

`defer func() { fmt.Println(i) }()`（测 ③ 输出 `100`）。这两种写法在“传参”和“捕获变量”上语义完全不同。

3.2.3. 回到你的这行代码

```
1   func TestKVBasic(t *testing.T) {
2     kv := KV{}
3     err := kv.Open()
4     assert.Nil(t, err)
5     defer kv.Close()          // ← 这一行
6     ...
7   }
```

Listing 14: 测试里的资源管理：Open 成功后立刻注册 Close

三点值得注意：

1. **位置放在 `Open` 成功之后**：顺序是“先确认拿到资源，再注册释放”。如果 `Open` 失败还去 `Close`，可能操作到半初始化的对象；
2. **用 defer 而不是直接调用**：中间的断言一旦失败（`assert.True` 返回 false 只是断言失败，但后续若用 `t.Fatal` 或 panic 就会提前退出），`Close` 仍然会被执行；
3. **将来这个 `Close` 会很要紧**：现在是内存实现、`Close` 什么都不做；等课程做到磁盘存储时，`Close` 要负责 **刷盘、关闭文件描述符**——那时漏掉一次 `Close` 就是数据丢失或句柄泄漏。`defer` 是这个保证的标准写法。

3.2.4. `defer` 的三个常见坑

坑一：循环里 defer——它推迟到**函数**返回才执行，不是循环体结束：

```
1   for _, name := range files {
2     f, err := os.Open(name)
3     if err ≠ nil {
4       return err
5     }
6     defer f.Close()   // × 所有文件都堆到函数结束才关，可能耗尽文件描述符
7   }
```

Listing 15: 循环里注册的 defer 会累积；正解是把循环体抽成独立函数，让每轮各自返回

坑二：defer 一个 nil 函数值——规范原文：`If a deferred function value evaluates to nil, execution panics w not when the "defer" statement is executed`

也就是 panic 发生在**函数返回时**，报错位置看起来会离现场很远。

坑三：以为 defer 很慢——那是旧印象。Go 1.14 起编译器对大部分 `defer` 采用 **open-coded**（把延迟调用直接内联到函数返回路径上），只有循环里或数量动态的情况才走较慢的栈上登记。日常在函数开头写几个 `defer`，开销可以忽略。

3.3. 陷阱与易错

1. **漏写括号**：`defer kv.Close` 编译不过（规范要求是“函数或方法调用”）；

2. **以为 defer 是“异步”**：它**不是**新开 goroutine，只是推迟到函数返回前，仍然在当前 goroutine 顺序执行（逆序）；
3. **在 defer 里修改命名返回值却不自知**：像测 ④ 那样 `defer func() { n++ }()` 会真的改变返回给调用方的值——这既是技巧（错误包装的常用手法），也是容易看漏的地方；
4. **把 defer 写在 `Open` 之前**：若 Open 失败，Close 会作用在未初始化的对象上。

3.4. 关键点

> 📌 **核心要点** `defer f()` = **把这次调用登记到“函数返回前”执行，逆序、参数即刻定格、panic 也照跑**。它是 Go 里管理资源（文件、锁、连接、测试清理）的标准写法：拿到资源后的下一行就写 defer。

3.5. 变式检验

1. **换成** `t.Cleanup(func(){ kv.Close() })`：测试专用的替代品，语义类似但**在整个测试函数（含子测试）结束后**执行，且与 `t.Run` 的子测试配合更好；
2. **换成“只读不写”的场景**：如果打开的是只读资源、且函数很短，直接顺序调用也常见——`defer` 的价值随“退出路径数量”增加而增加；
3. **换成跨函数传递**：`defer` 只作用于**当前函数**；想跨函数保证释放，要么把 cleanup 函数返回给调用方，要么用 `runtime.AddCleanup`（Go 1.24+ 的终结器替代品）。

3.6. 同类归类

1. 与 Python 的对照：Python 用 `with` 语句 + 上下文管理器（`__enter__` / `__exit__`），退出时机由缩进块决定；Go 的 `defer` 绑定的是**函数返回**，粒度更粗但更简单，不需要额外协议；
2. 与 C 的对照：C 靠 `goto cleanup` 集中释放（Linux 内核风格），是手工版的 defer；Rust 用 RAII（析构函数）自动释放，编译期确定；Go 选了“显式 defer + 运行时保证”这条中间路线；
3. 往下延伸：本课程后面打开日志文件、SSTable 文件时会大量用到 `defer file.Close()`；而“锁”也是同一模式——`mu.Lock(); defer mu.Unlock()` 几乎是最常见的 Go 并发写法。

3.7. Go 语言机制补充

3.7.1. defer 执行时机的精确定义

规范那句 “invoked immediately before the surrounding function returns” 配合同一段的补充说明，可以得到完整时序：

函数内的时刻	发生了什么
执行到 `return x`	先给返回值赋值（如果是命名返回值，此时已写入）
返回值就绪后、真正返回前	按**逆序**依次执行所有 defer——所以它们**能修改**返回值
goroutine panic 时	同样会执行 defer（recover 必须写在 defer 里才能拦住 panic）

这也解释了测 ④ 为什么输出 `11`：`return 10` 先把 10 写进命名返回值 `n`，然后 defer 把 `n` 加一，最后才真正返回。

3.7.2. defer、panic、recover 三者的关系

1. `panic` 沿调用栈向上传播，**每一层的 defer 都会被逐一执行**；
2. `recover()` **只在 defer 内调用才有效**，它中止 panic 的传播并返回 panic 的值；
3. 因此标准模式是：`defer func() { if r := recover(); r != nil { … } }()`。

> 📌 **核心要点** 一句话收束：**`defer kv.Close()` 是“不管函数怎么结束都保证执行这次清理”的登记动作。** 记住三点：推迟到**外层函数**返回、参数**立刻求值**、多个 defer **逆序**执行。

Index of Listings

Listing 1 能编译，但 ok 永远是零值（false） 3
Listing 2 正确写法：双值接双值；err 保持零值 nil 3
Listing 3 第 19 行正是 assert.True(t, string(val) == “v1” && ok && err == nil)——ok 为 false，断言失败 3
Listing 4 INDEX.md 由脚本扫描各 .typ 的一级标题自动生成，追加新问答后重新跑一次即可刷新 4
Listing 5 同一个索引表达式的两种写法 5
Listing 6 后缀 `_faststr` 是编译器为 map[string]X 生成的特化版本 5
Listing 7 单值形式没有独立实现——查找逻辑完全相同，差别只在要不要那个 bool . . 5
Listing 8 六步主干：空检查 → 并发检查 → 小 map 快路径 → 算哈希 → 取指纹 → 组内比对 6
Listing 9 看起来合理，但编译失败 7
Listing 10 return 只拿到 1 个值——comma-ok 的“特例形式”不包括 return 语句 7
Listing 11 绕一步即可——comma-ok 在 := 里合法 7
Listing 12 四段探针：顺序、参数求值、闭包、命名返回值 11
Listing 13 五条结论一次跑齐；②与③的差别是 defer 最经典的坑 11
Listing 14 测试里的资源管理：Open 成功后立刻注册 Close 12
Listing 15 循环里注册的 defer 会累积；正解是把循环体抽成独立函数，让每轮各自返回 12